

Python Functions Complete Tutorial

1. What is a Function?

A function is a reusable block of code that performs a specific task.

Instead of writing the same code again and again, we put it inside a function and call it whenever needed.

Example real-life idea:

“Make tea” is a function → you can call it multiple times

Functions improve reusability, readability, and modularity

2. Function Declaration vs Function Definition (Important Concept)

In Python, function declaration and function definition happen together.

Unlike languages like C/C++, Python does NOT support separate function declarations.

In simple words:

Declaration → Tells Python a function exists

Definition → Contains the actual code of the function

In Python:

When you define a function, you automatically declare it. No Separate Declaration is allowed

```
'''def function_name(parameters):  
    function body  
    statements  
    return value    # optional
```

Explanation

def → keyword used to define a function

function_name → name of the function

parameters → input values (optional)

: → indicates start of function body

Indentation → very important in Python

return → sends a value back (optional)

```
'''
```

3. Simple Function (No Parameters)

```
def greet():  
    print("Hello, Welcome to Python Functions!")  
  
a=greet()  
print(a)
```

```
Hello, Welcome to Python Functions!  
None
```

4. Function with Parameters

```
def add(a, b):  
    result = a + b  
    print("Sum:", result)  
print(add(4,5))
```

Sum: 9
None

5. Function with Return Statement

```
def multiply(x, y):  
    return x*y  
  
a=multiply(3,4)  
print(a)  
print(a*2)  
  
print(multiply(3,4))
```

12

6. Function with Multiple Return Values

```
def calculate(a, b):  
    return (a+b, a-b, a*b, (a,b))  
sum_, diff, prod, val = calculate(10, 5)  
print(sum_, diff, prod, val)
```

15 5 50 (10, 5)

Note:

Prints are always directed to console and are not used in calculations

return values are captured in the calling code and are used in calculations

7. Function with Multiple Return Values

#There are mainly two types of scope related to functions:

#Local Scope:

#Variables defined inside a function exist only inside that function.

#They cannot be accessed outside the function.

#Global Scope:

#Variables defined outside any function are global.

#Accessible inside and outside functions.

```
def my_func():
    x = 10 # local variable
    print("Inside function:", x)

my_func()
#print(x) # ❌ Error: x is not defined
```

```
x = 20 # global variable
```

```
def my_func():
    print("Inside function:", x)
```

```
my_func()
print("Outside function:", x)
```

```
Inside function: 10
Inside function: 20
Outside function: 20
```

8. Modifying global variables

#Without Global Keyword a new local variable is created inside function

```
x = 5
```

```
def change_global():
    x = 10
    print(x)
```

```
change_global()
print(x) # 10
```

```
10
5
```

```
#With Global Keyword global value is accessed and modified
```

```
x=5
```

```
def change_global():
    global x
    x = 10
    print(x)
```

```
x=20
```

```
change_global()
print(x) # 10
x=20
print(x)
```

```
10
10
20
```

9. Function Arguments in Python

```
# 9a.Positional Arguments:
```

```
# Arguments are assigned based on their position.
```

```
# Must match the order of parameters.
```

```
def greet(name, age):
    print(f"Hello {name}, you are {age} years old.")
```

```
greet("Alice", 25)
```

```
greet(25, "Alice") #Wrong Order causes confusion in Output
```

```
Hello Alice, you are 25 years old.
Hello 25, you are Alice years old.
```

9b. Keyword Arguments

#Arguments are assigned by parameter name, ignoring position.

#Improves readability.

```
def greet(name, age):  
    print(f"Hello {name}, you are {age} years old.")  
  
greet(age=25, name="Alice") # Hello Alice, you are 25 years old.
```

```
Hello Alice, you are 25 years old.
```

#9c Default Arguments:

#Parameters can have default values.

#Optional when calling the function.

```
def greet(name, age=18):  
    print(f"Hello {name}, you are {age} years old.")  
  
greet("Ahmed")  
greet("Alice", 25)
```

```
Hello Ahmed, you are 18 years old.
```

```
Hello Alice, you are 25 years old.
```

#9d Variable-length Arguments

#Used when you don't know how many arguments will be passed.

#*args (Non-keyword variable-length arguments)

#Groups arguments into a tuple.

```
def add(*numbers):  
    total=0  
    for i in numbers:  
        total+=i  
    print("Sum:", total)
```

```
add(1, 2, 3)  
add(4, 5, 6, 7, 8)
```

```
Sum: 6
```

```
Sum: 30
```

```
#9e **kwargs (Keyword variable-length arguments)
#Collects extra keyword arguments into a dictionary.
```

```
def show_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

show_info(name="Alice", age=25, city="Paris")
```

```
name: Alice
age: 25
city: Paris
```

```
#Future Goals: Explore: Keyword-only Arguments, Positional Only Arguments
```

10.Call by Object Reference (Call by Sharing)

In Python:

The reference to the object is passed to the function

The function and caller share the same object

Behavior depends on whether the object is mutable or immutable

```
#Immutable objects: int, float, str, tuple
#Cannot be changed inside function.
def modify(x):
    x += 5
    print("Inside:", x)

a = 10
modify(a) # Inside: 15
print(a)  # Outside: 10
```

```
Inside: 15
10
```

```
#Mutable objects: list, dict, set
#Can be modified inside function.
def modify(lst):
    lst.append(4)
    print("Inside:", lst)

numbers = [1, 2, 3]
modify(numbers)
print(numbers) # [1, 2, 3, 4]
```

```
Inside: [1, 2, 3, 4]
[1, 2, 3, 4]
```


11.Function returning a list, tuple and dictionary

```
def process_student_result(student_id, name, marks):
    # marks is a dictionary: subject -> marks

    total_marks = sum(marks.values())
    subjects = list(marks.keys())
    percentage = total_marks / (len(marks) * 100) * 100

    # Dictionary: student summary
    summary = {
        "id": student_id,
        "name": name,
        "total_marks": total_marks,
        "percentage": round(percentage, 2)
    }

    # List: subject-wise grades
    grades = []
    for subject, score in marks.items():
        if score >= 85:
            grades.append(f"{subject}: A")
        elif score >= 70:
            grades.append(f"{subject}: B")
        else:
            grades.append(f"{subject}: C")

    # Tuple: overall result stats
    result_stats = (
        len(marks),          # number of subjects
        max(marks.values()), # highest marks
        min(marks.values())  # lowest marks
    )

    return summary, grades, result_stats
```

```
marks_data = {
    "AI": 88,
    "Data Structures": 76,
    "Python": 91
}

summary, grades, stats = process_student_result(
    "AI-301",
    "Aisha Khan",
    marks_data
)

print("Summary:", summary)
print("Grades:", grades)
print("Stats:", stats)
```

```
Summary: {'id': 'AI-301', 'name': 'Aisha Khan', 'total_marks': 255, 'percentage': 85.0}
Grades: ['AI: A', 'Data Structures: B', 'Python: A']
Stats: (3, 91, 76)
```

12.Nested Function

```
def outer(a):
    print(a)
    b=6
    def inner(a):
        return a*b
        c=8
    result=inner(a)
print(outer(5))
print(inner(4))
```

```
5
None
NameError: name 'inner' is not defined
```

```

def process_transaction(account_number, balance, transaction_type, amount):
    print(f"Processing transaction for Account: {account_number}")

    def calculate_new_balance(balance, transaction_type, amount):
        fee = 0

        if transaction_type == "withdraw":
            fee = 50
            if amount + fee > balance:
                return None, "Insufficient balance"
            return balance - (amount + fee), "Withdrawal successful"

        elif transaction_type == "deposit":
            return balance + amount, "Deposit successful"

        else:
            return None, "Invalid transaction type"

    new_balance, message = calculate_new_balance(balance, transaction_type, amount)

    if new_balance is None:
        return {
            "account": account_number,
            "status": "Failed",
            "reason": message
        }

    return {
        "account": account_number,
        "status": "Success",
        "transaction": transaction_type,
        "amount": amount,
        "new_balance": new_balance
    }

```

```

result = process_transaction(
    account_number="ACC-101",
    balance=5000,
    transaction_type="withdraw",
    amount=2000
)

print(result)

```

```
Processing transaction for Account: ACC-101
```

```
{'account': 'ACC-101', 'status': 'Success', 'transaction': 'withdraw', 'amount': 2000,  
'new_balance': 2950}
```

13. Map In Python

```
#What is map() in Python?
```

```
#map() applies a function to every item of an iterable and returns a map object  
(iterator).
```

```
#Syntax:
```

```
#map(function, iterable)
```

```
#function → operation to perform
```

```
#iterable → list, tuple, set, etc.
```

```
#Output → map object (convert to list/tuple if needed)
```

```
#Example Without Map:
```

```
marks = [68, 72, 85, 90]
```

```
new_marks = []
```

```
for m in marks:
```

```
    new_marks.append(m + 5)
```

```
print(new_marks)
```

```
[73, 77, 90, 95]
```

```
#With Map
```

```
marks = [68, 72, 85, 90]
```

```
def add_grace(m):
```

```
    return m+10
```

```
new_marks = list(map(add_grace, marks))
```

```
print(new_marks)
```

```
## 14. Filter in Python
```

```
[78, 82, 95, 100]
```

#filter() selects elements from an iterable for which a given function returns True.

#When to use filter()?

#You want to remove unwanted data

#You want to select only valid items

#You want to apply conditions without loops

```
# Without filter()
students = [3.2, 3.8, 2.9, 3.6, 4.0]

eligible = []
for cgpa in students:
    if cgpa >= 3.5:
        eligible.append(cgpa)

print(eligible)
```

```
[3.8, 3.6, 4.0]
```

```
#With Filter:
students = [3.2, 3.8, 2.9, 3.6, 4.0]

def is_eligible(cgpa):
    return cgpa >= 3.5

eligible = list(filter(is_eligible, students))
print(eligible)
```

```
[3.8, 3.6, 4.0]
```

15. Lambda Function

#A lambda function is an anonymous one-line function used for short operations.

#Mostly when the task can be accomplished in a single line, for a short Logic

#Exp: Finding Even Numbers, Square of Numbers in a list etc.

```
#Example: Without lambda: Find all even numbers above 50
nums=[1,56,48,76,80,12,13]
def my_func(num):
    return num%2==0 and num>50

even_above_50=list(filter(my_func, nums))
print(even_above_50)
```

```
[56, 76, 80]
```

```
#Example: With Lambda: Find all even numbers above 50
even_above_50=list(filter(lambda x: x%2==0 and x>50, nums))
even_above_50
```

```
[56, 76, 80]
```

```
##Stand Alone lambda
#You can assign it to a variable and use it like a normal function:

# Single-line addition
add = lambda a, b: a + b
print(add(4, 5)) # 9

# Max of two numbers
maximum = lambda a, b: a if a > b else b
print(maximum(10, 15)) # 15
```

```
9
15
```

16. Reduce

```
#reduce() applies a function cumulatively to the items of an iterable,  
#reducing it to a single value.
```

```
#It always takes two values at a time
```

```
#Part of the functools module
```

```
#Often combined with lambda
```

```
#How it works
```

```
    #Take first two elements → apply function → get result
```

```
    #Apply function on result and next element
```

```
    #Continue until all elements are consumed
```

```
## Numbers: 1, 2, 3, 4
```

```
# Function: add
```

```
# Steps:
```

```
# step1: 1 + 2 = 3
```

```
# step2: 3 + 3 = 6
```

```
# step3: 6 + 4 = 10
```

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
total = reduce(lambda a, b: a + b, numbers)
```

```
print(total)
```

15

```
words = ["Python", "is", "fun"]
```

```
sentence = reduce(lambda a, b: a + " " + b, words)
```

```
print(sentence) # "Python is fun"
```

Python is fun

Python Functional Functions Comparison

Feature	map()	filter()	reduce()
Output	Same size ≤ input size		Single value
Operation	Transform	Select	Aggregate
Function args	1	1	2

17. Decorators

```
'''Decorator: In Simple Terms: A function which takes another function updates it in some way and returns a modified function called decorated function'''
```

```
def greet(f):  
    def modified_func():  
        print("Good Morning")  
        f()  
        print("Bye Bye!!")  
    return modified_func
```

```
#One Method to add decorator  
@greet  
def hello():  
    print("Hello!")  
  
hello()
```

```
Good Morning  
Hello!  
Bye Bye!!
```

```
#Another Method  
greet(hello)()  
#Comment the decorator above
```

```
Good Morning  
Good Morning  
Hello!  
Bye Bye!!  
Bye Bye!!
```


18. Generators

'''

Generator allows you to create an iterable sequence of values

It returns a generator object which can be used to generate the values one by one

Think of a generator like a magic box:

You ask it for the next item.

It gives you one thing at a time.

It remembers where it left off so it can continue.

It uses yield instead of return in a function

```
def simple_gen():  
    yield 1  
    yield 2  
    yield 3
```

'''

Here, simple_gen is a generator function because it uses yield.

yield = "give this value, pause here, and come back later."

return = "give this value and finish forever."

'''

```
g = simple_gen() # Create the generator
```

```
print(next(g)) # 1
```

```
print(next(g)) # 2
```

```
print(next(g)) # 3
```

#Instead of giving all values at a time, it is giving values one by one

```
1  
2  
3
```

```
#Iterating Generator with a Loop
for value in simple_gen():
    print(value)
```

```
1
2
3
```

```
'''
Why use generators?

If you have a huge list, storing it all is memory-heavy.
Generators only give one item at a time, saving memory.
They can even produce infinite sequences because they don't store everything. '''
```

```
def infinite_numbers():
    n = 0
    while True:
        yield n
        n += 1
```

```
for num in infinite_numbers():
    if num > 10:
        break
    print(num)
```

```
0
1
2
3
4
5
6
7
8
9
10
```

#Note:It never creates all numbers, just gives the next one on demand.